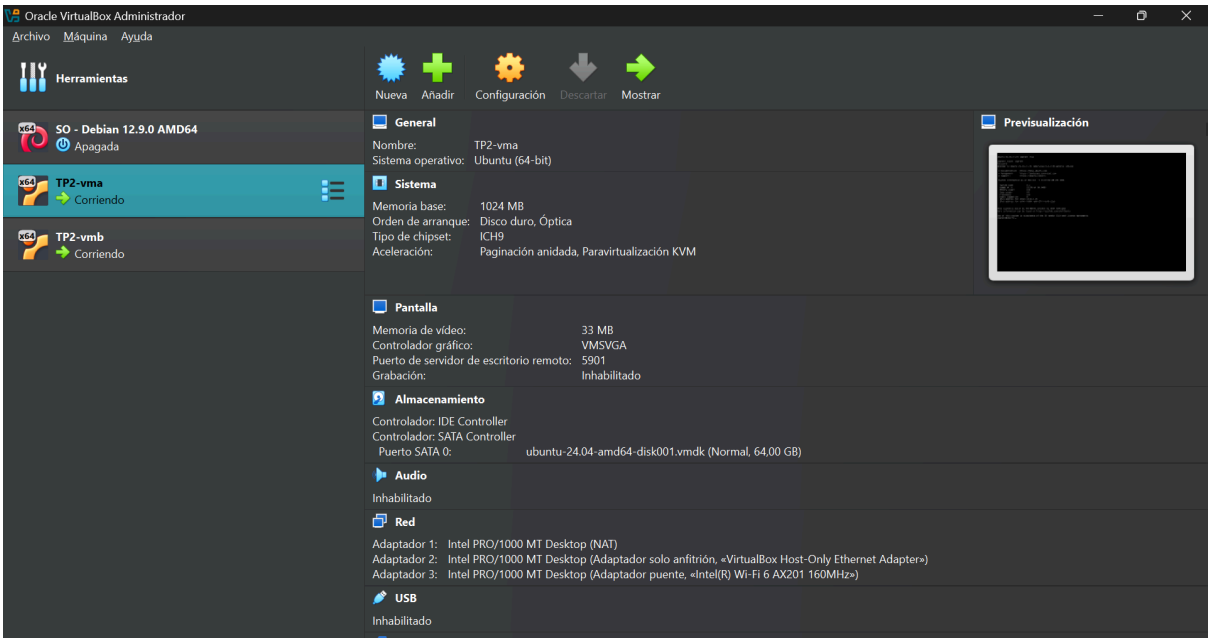
Programación Distribuida y Tiempo Real

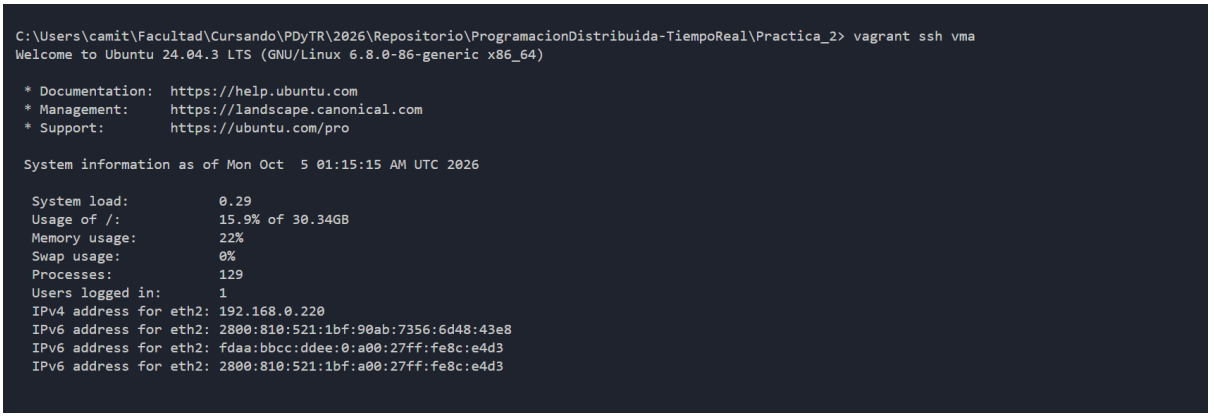
Práctica 2

Integrantes: Juan Manuel Tarrío (23990/6) - Camila Florencia Tarrío (21016/6)

1. La versión de Ubuntu LTS que elegimos es **bento/ubuntu-24.04**
    - **Características:**
        i. Versión: 202510.26.0
        ii. Origen: proyecto Bento, mantenido por Chef (Progress Software). Es un tercero, no Canonical. Canonical dejó de publicar boxes de Vagrant a partir de Ubuntu 24.04, después de que Vagrant cambiara su licencia a Business Source License. Por eso no hay una box oficial de 24.04 y usamos la de un tercero.
    - **Diferencias:**
        i. Cambia el publicador de la box (de Canonical a Bento) y las versiones del sistema, pero para los experimentos de comunicación con sockets no hay diferencias funcionales.
    - **Análisis con IA (Claude (Anthropic)):**
        i. Para esta práctica, 24.04 LTS es la mejor opción. Es madura, tiene soporte hasta 2029 y su box de terceros está bien mantenida. 22.04 solo sería preferible si la prioridad fuera usar una box oficial. 26.04 conviene recién cuando tenga más recorrido. En cualquier caso, lo que más importa para la validez de los experimentos es fijar la versión de la box y que todos usen la misma.

    Capturas de pantalla solicitadas (identificación en VirtualBox e inicio de la terminal en funcionamiento):







2. **<u>Explicación del experimento</u>:**
   Para la medición de tiempos se decidió repetir el algoritmo con la misma cantidad de datos 100 veces, con el fin de encontrar el caso mínimo, máximo y promedio.
   Se utilizan 5 repeticiones de calentamiento que no se tienen en cuenta para las mediciones con el fin de no ensuciar los tiempos con los primeros casos. Estos suelen tardar más por varios motivos (ventana de congestión TCP, autoajuste del buffer, cache de instrucciones y predicciones de saltos).
   Tanto el read() como write() del lado del cliente se encuentran aislados en la medida del tiempo dejando fuera cualquier lógica que afecte a la medición.
   <u>Pseudocódigo</u>:

   inicio del timer
   variable = función rea d() / write();
   fin del timer

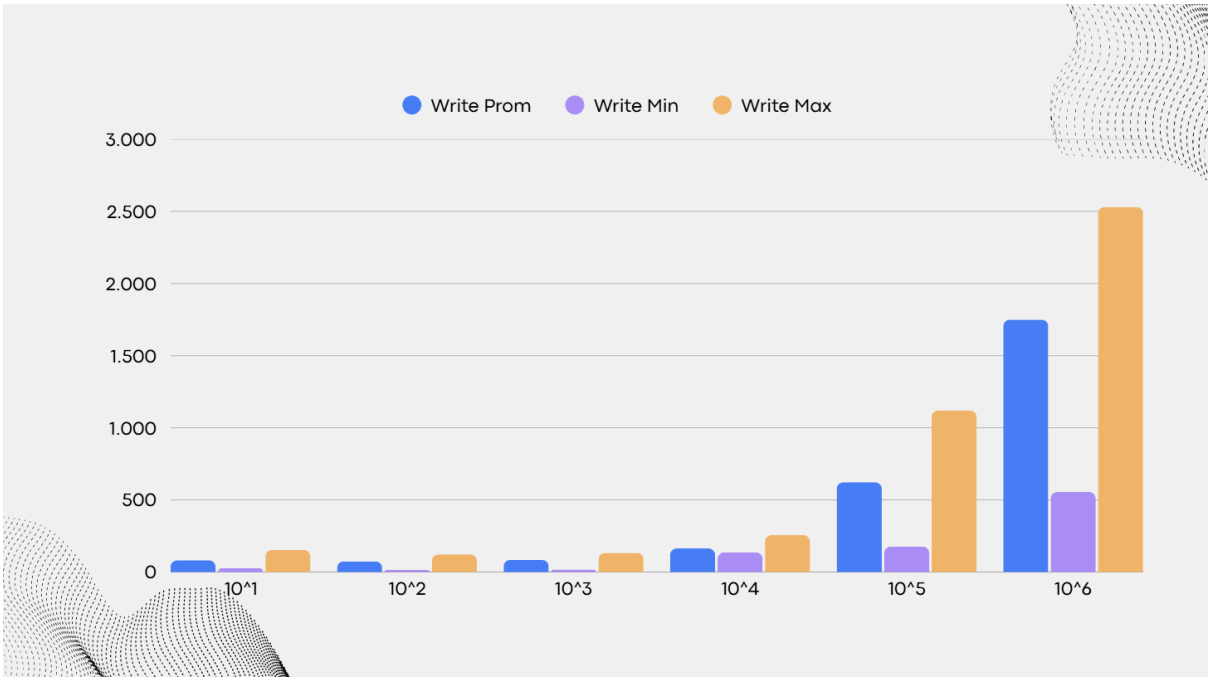
   La variable sirve para controlar lo que llega o se envía en ambos casos de corte, tanto si la función devuelve 0 (fin de comunicación) como si retorna -1 con errno seteado.

EJERCICIO 2 - TIEMPOS MEDIDOS EN EL CLIENTE (nanosegundos)

BYTES	WRITE PROM	WRITE MIN	WRITE MAX	READ PROM	READ MIN	READ MAX
10	81531	26055	153800	15954160	8969745	31784413

100	73738	14259	122557	15997625	9395120	33842719
1000	83812	16834	132675	16323856	9783222	31721760
10000	163785	136305	255921	17193853	13180937	47062283
100000	622436	176049	1122145	27594660	20493328	51478874
1000000	1749373	554912	2530022	165888488	146278480	202794293

WRITE(): La sentencia write() se encarga de mover los datos desde el arreglo en memoria de usuario hacia el buffer de envío del socket, dentro del kernel. No espera a que los datos salgan ni a que lleguen (no bloqueante).



- Primeros 3 tamaños (10,100,1000): Hay algunas variaciones en los mínimos, pero no son atribuibles al tamaño sino al ruido del muestreo. El promedio de tiempos son casi los mismos, esto se debe a la baja cantidad de bytes enviados, el mayor tiempo de procesamiento se lo lleva la system call.

- 10.000 bytes: Primer cambio en el promedio de los tiempos, crece casi el doble. El tiempo corresponde mayormente a la transferencia de memoria desde el Espacio de usuario de kernel y a la asignación de las estructuras internas del kernel. Trabajando con esta cantidad de datos el trabajo es “determinístico”. Es suficientemente grande para que el ruido del planificador no lo domine y suficientemente chico como para no chocar con algún límite del buffer.

- 100.000 bytes: Viendo el promedio se nota como sube ~4 veces más que el anterior. Lo más interesante de este tamaño es el mínimo, el cual es muy parecido a trabajar con 10k (conclusión: en algunos casos trabajar con 10k

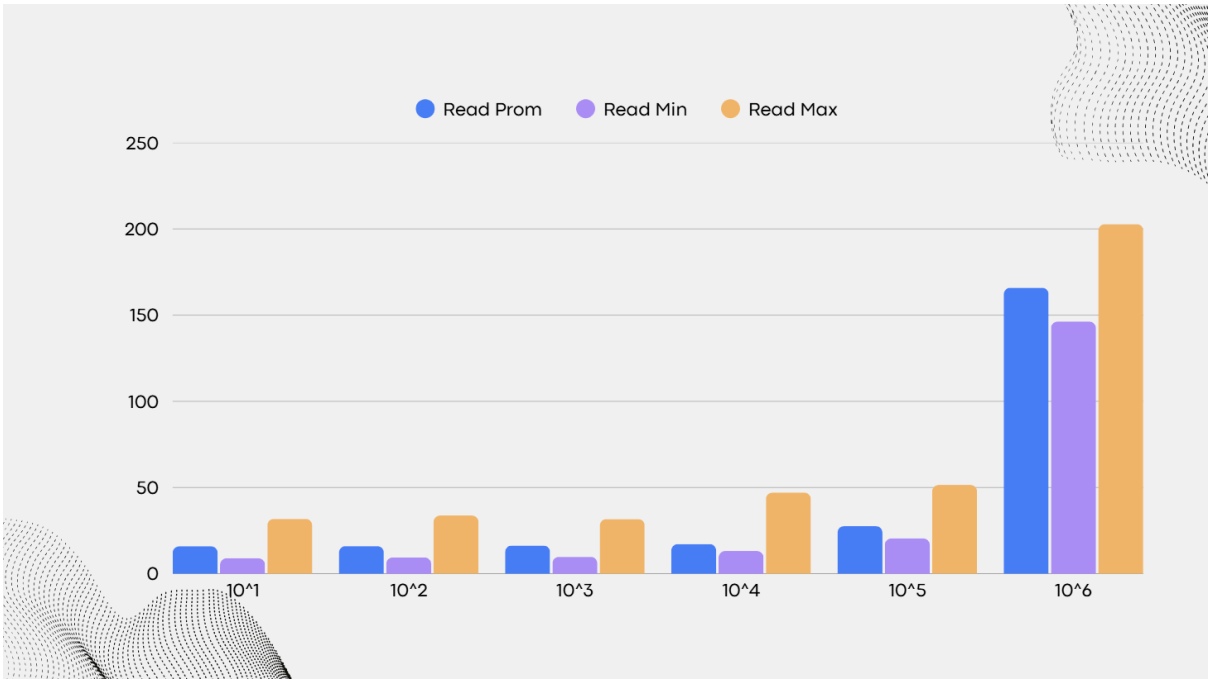
es lo mismo que trabajar con 100k). Además la dispersión explota, del mínimo 176 microsegundos a 1,12 milisegundos haciendo una distribución muy asimétrica. ¿Por qué pasa esto? Lo más probable es que en esos ~176 nanosegundos se produzca alguna interrupción sumando esos cientos de microsegundos afectando al promedio. La mayoría de las mediciones están cerca del coste real, y algunas se ven castigadas por las interrupciones.

- 1.000.000 bytes: En este caso el millón de bytes entra directamente en el buffer del socket, esto porque linux ajusta el tamaño dinámicamente. En caso de no ajustarlo hubiera tardado 200 veces más ya que tendría que haber un while de por medio y esperando a que la capa transporte, mediante el protocolo TCP, envíe los paquetes liberando el buffer para poder ingresar los nuevos datos.

READ(): Genera una syscall que mueve los datos del espacio de recepción del socket (espacio kernel) al buffer del programa (espacio usuario).

Parece una simple función pero atrás hay un proceso grande, luego de hacer la sentencia write() se queda esperando en la sentencia read(), por lo tanto:

1. cuenta lo que tarda en transferir los datos del write() a través de la capa de transporte con el protocolo TCP,
2. cuenta el recibimiento de los mismos de parte del servidor con su propio read(),
3. el envío de la respuesta del servidor hasta que se deposita en el espacio de recepción del kernel y se realiza la syscall que dispara el read() moviendo la respuesta del servidor del espacio de recepción del socket al buffer del programa.



- Los primeros 3 tamaños (10,100,1000): tardan ~16 milisegundos, esto representa el costo fijo del enlace, dentro se encuentran las 2 transferencias de datos por la capa de transporte, la placa virtualizada de la VM y el procesamiento de las syscalls de ambas computadoras. El tiempo de transmisión no es lo suficiente como para afectar a los tiempos, todo lo que se mide aca es el RTT, el tiempo que tarda 1 paquete en ir y volver.

- 10.000: Tomando a los ~16 milisegundos como el piso, con este tamaño se ve un +1,24 milisegundos por encima del piso, el volumen de datos deja una marca visible. Con este tamaño el envío no puede hacerse en 1 solo paquete TCP (~1460 bytes), se debe fragmentar en ~7 segmentos. Igualmente el piso sigue representando un 93% del total.

- 100.000: Con este tamaño ya el piso representa un 58% del total, el otro 42% representa a la transmisión que es proporcional al volumen de datos. Los datos (100.000 bytes) tienen que dividirse en ~69 segmentos de ~1460 bytes y cada paquete tiene que: ganar el acceso al canal, ser transmitido y recibir la confirmación de nivel de enlace (ACK).

- 1.000.000: Con este tamaño se reduce significativamente el porcentaje que ocupa la latencia, pasa a un 9,6%, el resto del tiempo representa al tiempo de transmisión de los ~685 segmentos, con respecto al ancho de banda (bytes / tiempo de transmisión) es 6,67 MB/s, un 22% menos que en 100.000 bytes esto quiere decir que la tasa efectiva cae en transferencias largas ya sea tanto por la cantidad de segmentos como los eventos de interferencia y retransmisiones que se pueden generar.

Descripción de las computadoras usadas:

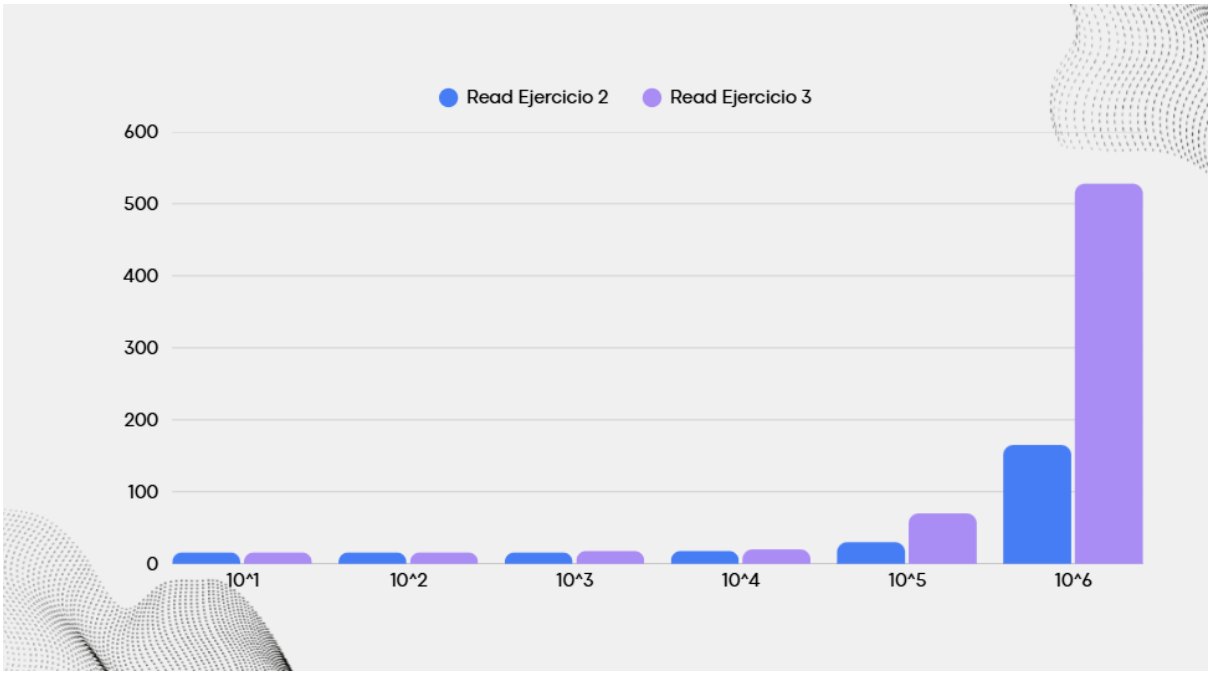
- **Cliente:**
  - CPU: Intel(R) Core(TM) i5-7200U (2 núcleos / 4 hilos)
  - RAM: 12 GB
  - OS: Ubuntu 24.04 LTS (Kernel 6.14.0-37-generic, 64-bit)
  - Red: misma red local

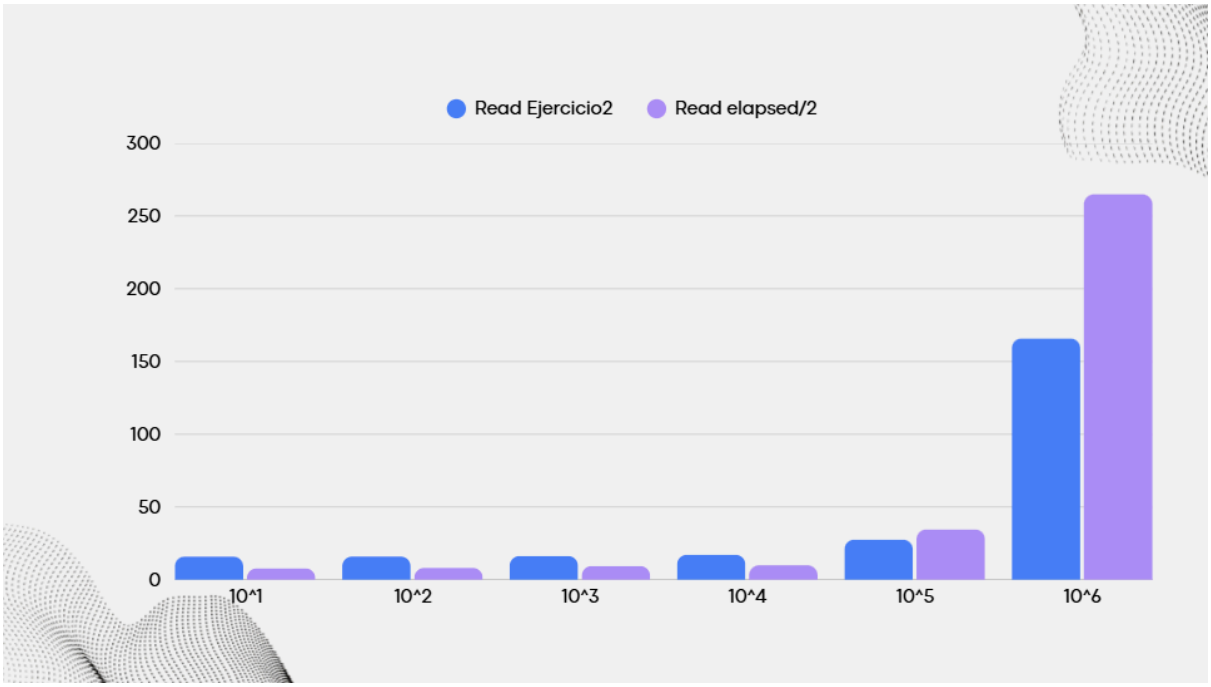
- **Servidor:**
  - CPU: 1 vCPU (VirtualBox)
  - RAM: 1024 MB
  - OS: Ubuntu 22.04.5 LTS (Kernel 5.15.0-190-generic, x86_64)
  - Red: misma red local (192.168.0.0/24), 192.168.0.250, bridged sobre WiFi

3. Cambios con respecto al ejercicio 2:
En el write() del servidor se agrega el chequeo de envío parcial ya que no se envía 1 solo byte como en el ejercicio anterior sino que se envía lo que recibe de parte del cliente. Del lado del cliente se modifica el read() ya que es necesario ejecutar el read() varias veces para poder leer todos los datos que se envían. Si dejamos 1 solo read() fuera de un loop while se leerían los 3 primeros casos bien, pero a partir del 10^4 se empiezan a leer los primeros paquetes que llegan a través de la capa de

transporte desde el servidor y no se leerían los nuevos paquetes que van entrando al buffer del kernel. En el caso de 10^6 esto generaría un error en el write() por parte del servidor ya que no se liberaría el buffer del kernel de parte del cliente haciendo que se llene y cortando la comunicación.
El ejercicio 2 no permite estimar el tiempo de comunicación en una dirección, porque su intercambio es asimétrico: N bytes de ida contra una cantidad fija de bytes de vuelta. En el ejercicio 3 al transmitir la misma cantidad de bytes en ambos sentidos, el tiempo total corresponde a 2 comunicaciones idénticas y su mitad es una estimación de lo que tarda 1 comunicación.

Unidad de medida= milisegundos





EJERCICIO 3 - TIEMPOS MEDIDOS EN EL CLIENTE (nanosegundos)

BYTES	WRITE PROM	WRITE MIN	WRITE MAX	READ PROM	READ MIN	READ MAX
10	89808	71103	147060	15661988	10111626	48646408
100	85761	37005	155849	16318351	9003821	46168175
1000	89256	69338	159727	18773401	9191600	92680106
10000	134205	103580	201219	19713946	10355639	40131167
100000	581646	143937	782392	68661873	42962460	201611869
1000000	1439844	952235	1778559	528330717	383173258	1444822390

Se observa como el write() del lado del cliente mantiene la misma lógica y tiempos que el ejercicio 2. Esto se debe a que no es bloqueante y el bloque se genera el read(), por lo tanto el write() solo se encarga de mover los datos del espacio de usuario al buffer del espacio del kernel.

La diferencia estructural (perteneciente al primer gráfico) entre ambos experimentos está en lo que el read() del cliente debe esperar. En el ejercicio 2 espera a que sus propios N bytes terminen de atravesar la red, a que el servidor los reciba completos, y al retorno de un valor fijo de byte de confirmación. En el ejercicio 3 espera además a que el servidor transmita N bytes de vuelta, con su propio tiempo de transmisión y su propia ventana de congestión. Ese es el motivo por el cual el tiempo total aproximadamente se duplica, y es precisamente lo que justifica dividir por dos para estimar el costo de una dirección.

Con respecto a los 4 primeros tiempos (pertenecientes al segundo gráfico) el RTT/2 del ejercicio 2 da ~8, por lo tanto para estos casos se estima que una comunicación en una dirección cuesta ~8 milisegundos. En los últimos 2 tamaños hay un residuo significativo:

- 10^5: ~15 milisegundos
- 10^6: ~107 milisegundos

Analizándolo con la IA llegamos a la conclusión:
Su origen más probable es el reinicio de la ventana de congestión por inactividad: en el ping-pong, el período entre transmisiones consecutivas en una misma dirección alcanza los 530 ms, superando el RTO mínimo de Linux (200 ms), lo que fuerza un arranque lento en cada iteración y reduce el ancho de banda efectivo de 6,7-8,6 MB/s a 3,8 MB/s por dirección.

4.

Parte A (2 máquinas virtuales):

EJERCICIO 4 - TIEMPOS MEDIDOS EN EL CLIENTE (nanosegundos)

bytes	write prom	write min	write max	read prom	read min	read max
10	277155	162413	1164386	680061	1942	1749032
100	272024	164688	1380017	644812	7284	1666613
1000	208024	170496	461581	497488	233712	924846
10000	674491	46001	28309678	2114284	9104	10645621
100000	730589	370753	3662826	2004518	236918	4750435
1000000	3634642	225630	16836836	14342145	4707006	27042433

Parte B (1 linux host y 1 máquina virtual):

EJERCICIO 4 - TIEMPOS MEDIDOS EN EL CLIENTE (nanosegundos)

bytes	write prom	write min	write max	read prom	read min	read max
10	67200	27906	662807	627134	48080	1837535
100	93515	29805	840248	594407	14738	2139285
1000	67751	29884	662325	530926	19079	1567486
10000	116091	45581	1179213	1083800	24138	5551566
100000	69952	50611	482299	753754	306640	2992791
1000000	411521	249951	1784859	6788502	4232979	12090682

Análisis con IA (Claude (Anthropic)):
El método más confiable es el ping-pong (medición de ida y vuelta, o RTT), hecho con los mismos procesos y protocolo que usará la aplicación:

1. Medir con un solo reloj. El cliente toma t0, envía N bytes, el servidor los recibe completos y devuelve los mismos N bytes, y el cliente toma t1 cuando recibe todo. El tiempo de un sentido se estima como (t1 − t0)/2. Así se evita depender de relojes de dos máquinas distintas, que nunca están perfectamente sincronizados.
2. Usar un reloj de alta resolución, que no se vea afectado por ajustes de la hora del sistema.
3. Repetir muchas veces (cientos o miles de iteraciones) y descartar las primeras como calentamiento: conexión TCP en slow-start, cachés frías, páginas de memoria sin mapear.
4. Reportar estadísticas robustas: mediana y percentiles, además del promedio, y también mínimo y máximo. La mediana es menos sensible a picos aislados.
5. Variar el tamaño de los mensajes (por ejemplo, de 10 a 10^6 bytes) para separar la latencia, que domina con mensajes chicos, del ancho de banda, que domina con mensajes grandes. Con el modelo $t(n) \approx \text{latencia} + n / \text{ancho de banda}$ se pueden estimar ambos parámetros.
6. Controlar el ambiente: red cableada en lugar de Wi-Fi, sin otro tráfico, y máquinas sin carga. Documentar hardware, sistema operativo y topología de red.
7. Complementar con herramientas estándar: ping como referencia de latencia de red e iperf3 para medir el ancho de banda máximo.

5. Los tiempos medidos no se ven afectados por el orden de ejecución ni por las diferencias de inicio entre los programas, porque el establecimiento de la conexión queda fuera de las regiones cronometradas. Un retardo de 10 segundos antes del listen() del servidor solo incrementa la duración total de la corrida: el cliente lo absorbe mediante su bucle de reintentos de connect() y las mediciones posteriores resultan indistinguibles de las de la corrida base.
La situación es radicalmente distinta si el retardo se ubica dentro de una región medida. Un sleep(10) entre el write() y el read() del cliente hace que la respuesta del servidor llegue al buffer de recepción antes de que se invoque read(), con lo cual la llamada retorna inmediatamente y el tiempo medido se reduce a microsegundos para cualquier tamaño de mensaje. La medición deja de reflejar la comunicación y pasa a reflejar únicamente el costo de una copia de memoria.
Este efecto se observa de forma espontánea en los propios resultados del ejercicio 4: en la Parte A, el mínimo de read() para 10 bytes es de 1.942 ns, tres órdenes de magnitud por debajo del promedio de 680.061 ns. Un valor así es incompatible con la espera de un ida y vuelta por red y solo se explica porque, en esas iteraciones, el cliente se demoró lo suficiente entre el envío y la recepción como para que la respuesta ya estuviera disponible.
La red host-only implementa íntegramente las capas de enlace, red y transporte: los paquetes atraviesan la pila TCP/IP completa de cada sistema operativo huésped, con segmentación según MSS, control de congestión y control de flujo idénticos a los de una red física. Lo que se sustituye es únicamente la capa física: en lugar de

un medio de transmisión, el conmutador virtual de VirtualBox copia las tramas entre las interfaces virtuales dentro de la memoria del anfitrión. En consecuencia, los tiempos medidos reflejan el costo del procesamiento de protocolos y de la virtualización, sin incluir el tiempo de transmisión ni la contención por el acceso al medio.